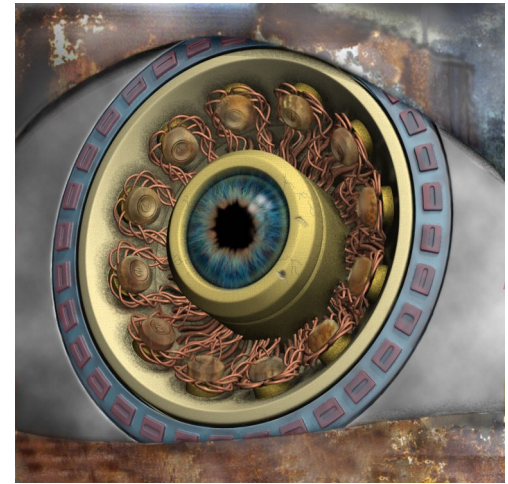
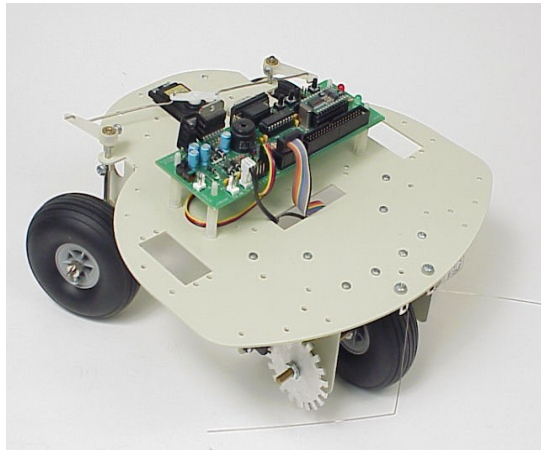




Clasificación y Segmentación de Imágenes Digitales

Robótica y Percepción Computacional

4º Curso. Graduado en Ingeniería Informática

Luis Baumela

Departamento de Inteligencia Artificial

Universidad Politécnica de Madrid



POLITÉCNICA

Clasificación y segmentación de imágenes digitales

1. Introducción

- Planteamiento del problema.
- ¿Qué es segmentación y clasificación?
- Solución propuesta.

2. Segmentación de imágenes digitales

3. Clasificación

Introducción

- **Objetivo**

Identificar en las imágenes líneas, flechas y fondo.



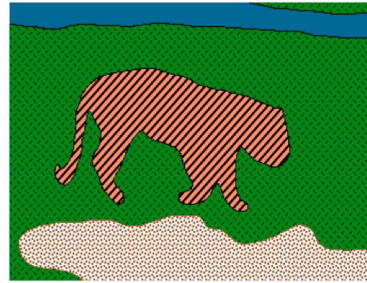
Proceso:

1. Qué píxeles pertenecen a cada región (segmentar)
2. Describir la región
3. Toma de decisiones: consigna movimiento, clasificar.

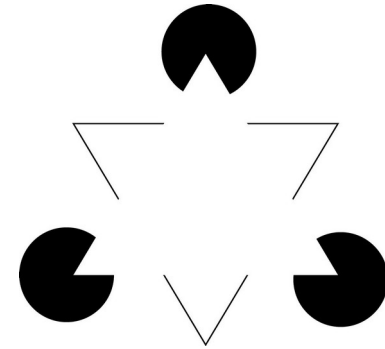
Introducción

- **Segmentación**

Agrupar los píxeles de una imagen en regiones que comparten alguna propiedad.



textura



Person
Bicycle
Background

semántica

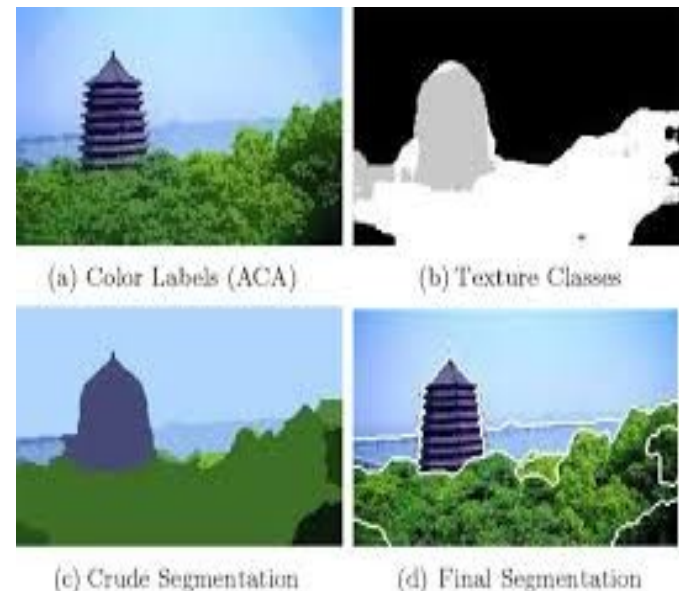
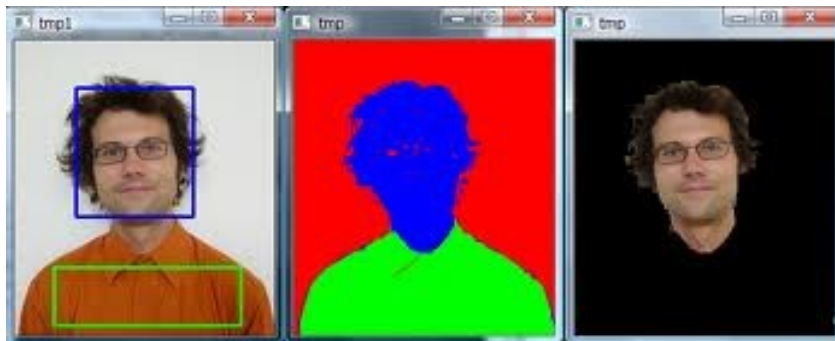


continuidad

Introducción

- **Segmentación**

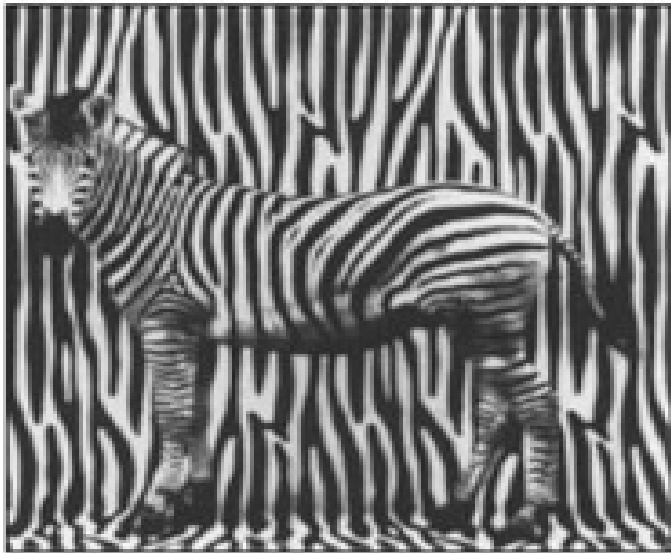
Cada píxel de la imagen tiene asociada una etiqueta que representa el segmento/región al que pertenece.



Introducción

- **Segmentación**

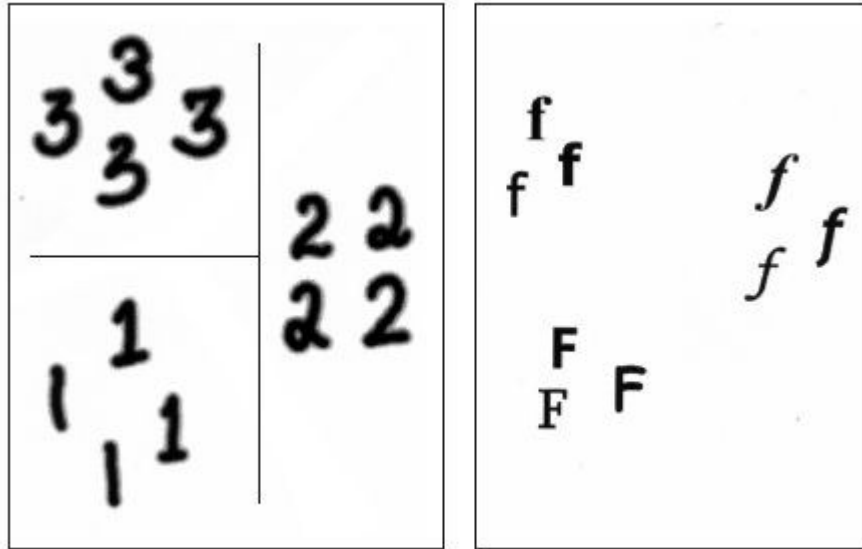
Es una representación más abstracta de la escena, paso intermedio para otros problemas como la reconstrucción, o el reconocimiento.



Introducción

- **Clasificación**

Asignación de una etiqueta a un dato.



Describimos el dato con un vector de m características discriminantes numéricas, $\mathbf{x} \in \mathbb{R}^m$.

Representamos las etiquetas mediante un conjunto discreto de valores $\{c_1, \dots, c_k\}$.

Introducción

- **Aproximación sugerida**

Describimos cada píxel de la imagen con un vector de características discriminantes, $\mathbf{x} \in \mathbb{R}^m$

$$\mathbf{x} = [x_i, y_i, I(x_i, y_i), |\nabla I(x, y)|, \underbrace{R, G, B, \dots}]$$



Introducción

- **Aproximación sugerida**

Describimos cada píxel de la imagen con un vector de características discriminantes, $\mathbf{x} \in \mathbb{R}^m$.

Sólo utilizaremos la apariencia (valores de los píxeles), aunque también puede utilizarse otros datos como movimiento, disparidad (profundidad), etc.

Sólo consideraremos segmentación bottom-up. Ignoramos la contribución del reconocimiento o la estructura.

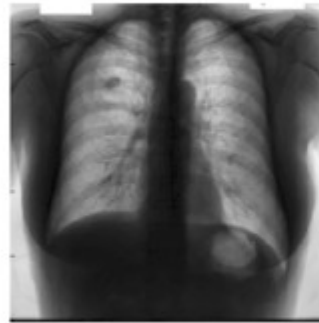
- **Segmentamos empleando un clasificador**

Segmentar $\equiv$ Etiquetar $\equiv$ Clasificar

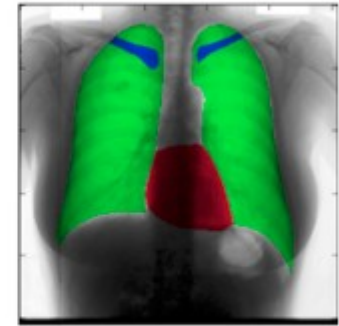
Segmentación

- ¿Para qué?

Localizar y describir
regiones de interés



Input Image



Segmented Image

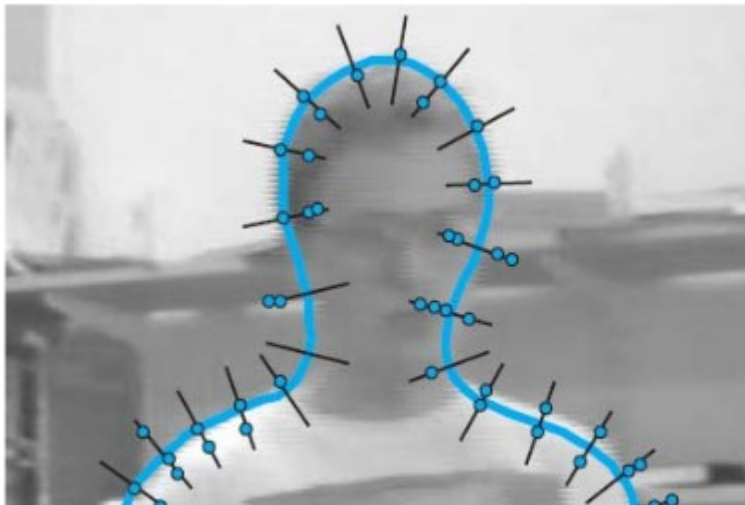
Proceso intermedio
para otras tareas de
más alto nivel:
reconstrucción,
análisis/interpretación.



Segmentación

- Aproximación basada en detección de bordes
Detecta discontinuidades en la descripción de la imagen para establecer las fronteras de las regiones.

Adecuado cuando la región tiene una frontera claramente definida.



Active contours (Isard, Blake, 1998)



Snakes (Kass, Witkin, and Terzopoulos 1988)

Segmentación

- Aproximación basada en detección de bordes

Morphological Snakes (P. Márquez-Neila, PAMI, 2014)

Algoritmo muy eficiente de evolución de curvas basado en operadores de morfología matemática.

```
skimage.segmentation.morphological_geodesic_active_contour(gimage, iterations,  
init_level_set='circle', smoothing=1, threshold='auto', balloon=0, iter_callback:  
<function <lambda>>)
```



Segmentación

- Aproximación basada en la agrupación

Mean-shift segmentation (Comaniciu, PAMI 2002)

Agrupar los píxeles en conjuntos homogéneos de forma no supervisada.

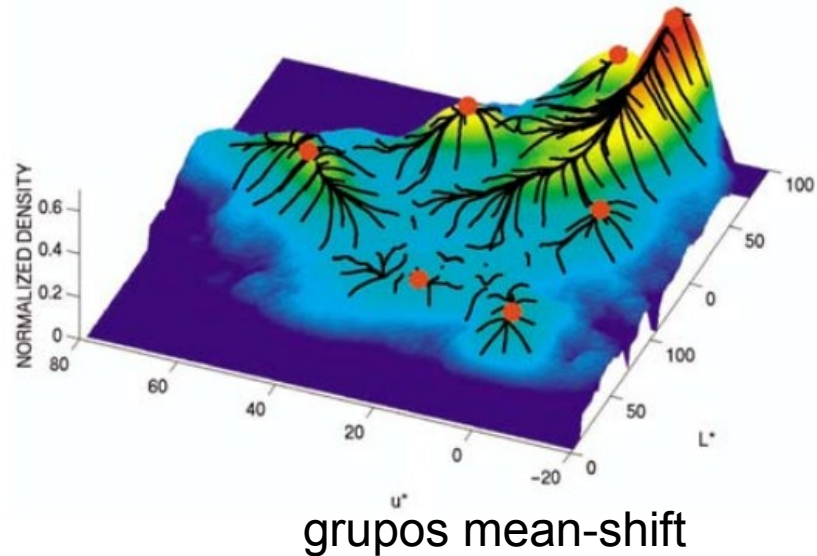
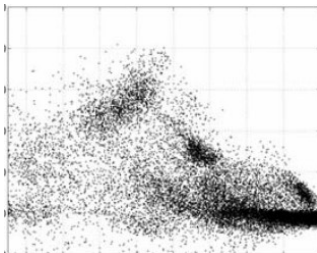
Cómputo alto. Resultados no deterministas.

```
class sklearn.cluster.MeanShift(*, bandwidth=None, seeds=None, bin_seeding=False, n_jobs=None, max_iter=300)
```

imagen



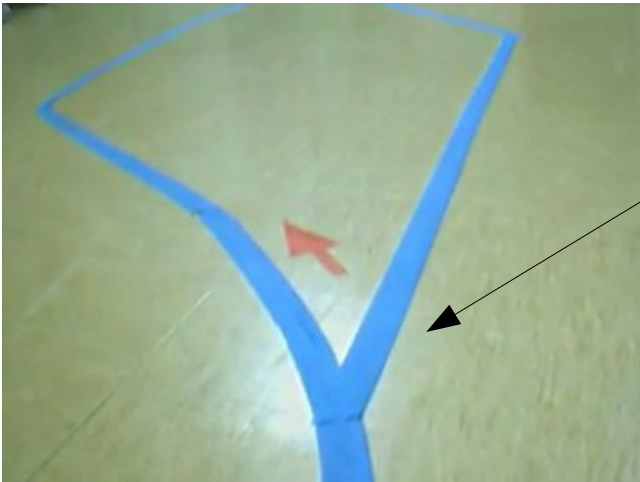
espacio color
normalizado



Segmentación

- Aproximación basada en regiones (**sugerida**)
Etiqueta y agrupa píxeles en regiones homogéneas de forma supervisada.

El descriptor de cada píxel serán sus colores



$$I(\mathbf{x}_i) = [r_i, g_i, b_i]$$

Entrenar un clasificador que etiquete cada píxel.

Clasificación

- **Tipos algoritmos**

Asignación de una etiqueta a un dato.

Describimos el dato con un vector de m características discriminantes numéricas, $\mathbf{x} \in \mathbb{R}^m$.

Representamos las etiquetas mediante un conjunto discreto de valores $\{c_1, \dots, c_k\}$.

- Clasificación supervisada

Para la construcción del clasificador se conocen un conjunto de objetos y sus etiquetas asociadas.

$$\mathcal{D} = \{(\mathbf{x}_1, c_1), \dots, (\mathbf{x}_n, c_k)\}$$

- Clasificación no supervisada

Sólo se conocen los objetos, sus etiquetas son desconocidas y serán el resultado del proceso de aprendizaje

$$\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$$

Clasificación Supervisada

- **Tipos de clasificadores**

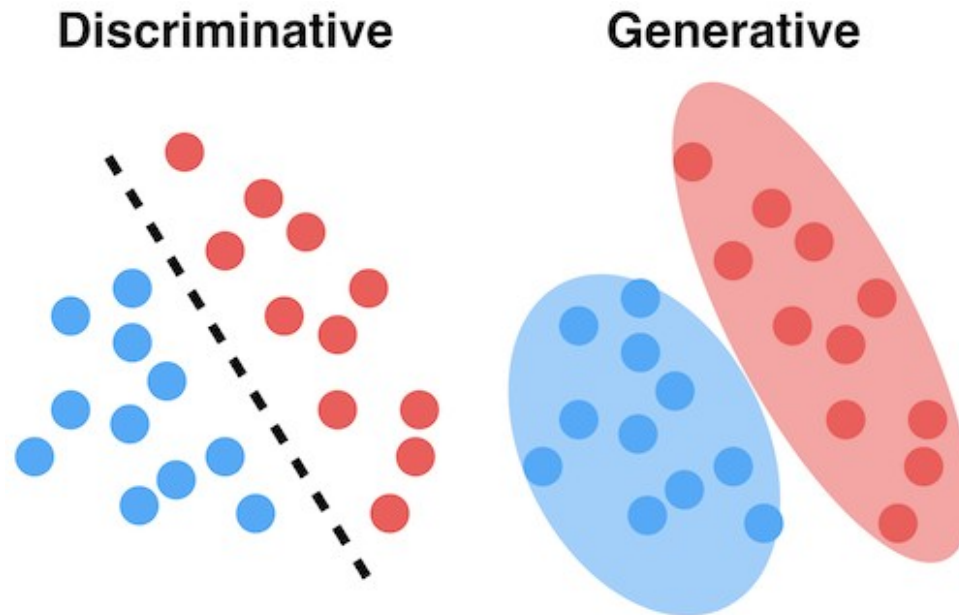
- Generativos

Modelan la distribución de las muestras dentro de las clases.

- Discriminativos

Modelan la frontera que separa las clases

- SVM



- Distancia euclídea
- K vecinos
- Gaussiano

Clasificación supervisada

Sean $\mathcal{D} = \{(\mathbf{x}_1, c_1), \dots, (\mathbf{x}_n, c_k)\}$ los datos de entrenamiento con etiquetas $C = \{c_1, \dots, c_k\}$. Nuestro objetivo es aprender un modelo g

$$c = g(\mathbf{x}|\mathcal{D})$$

que prediga la etiqueta de cualquier objeto nuevo $\mathbf{x}$.

En nuestro caso, los objetos a clasificar son los píxeles de la imagen.

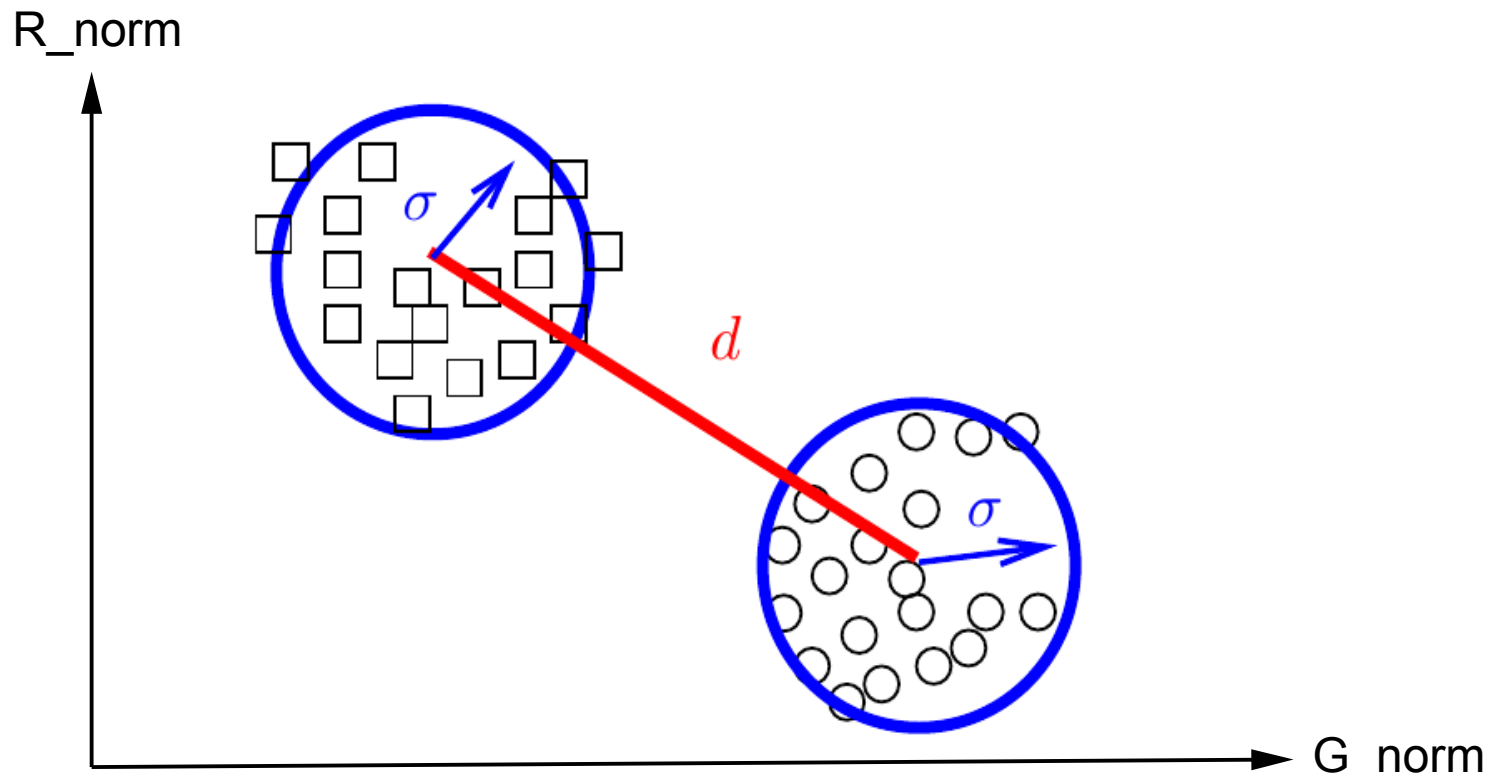
• Clasificador de la distancia euclídea

- Representa cada clase mediante el centro de masas de los datos de entrenamiento
- El modelo g mide la distancia de una muestra al representante de cada clase
- Asigna una muestra desconocida a aquella clase cuyo representante sea el más cercano

Clasificador de la distancia euclídea

■ Hipótesis.

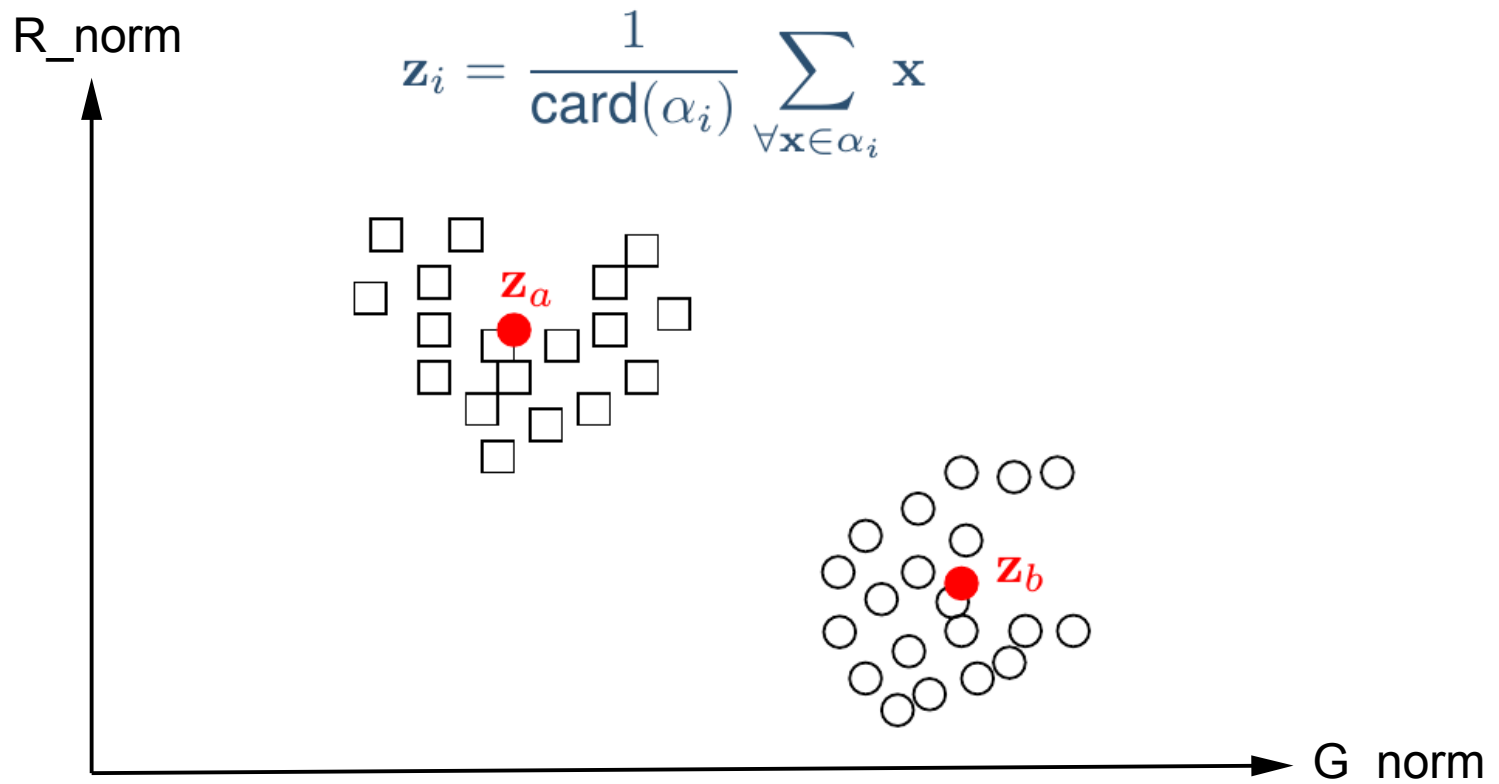
- ◆ La dispersión de las clases es pequeña en relación a la distancia entre ellas. Es decir, $d \gg \sigma$



Clasificador de la distancia euclídea

■ Hipótesis.

- ◆ La dispersión de las clases es pequeña en relación a la distancia entre ellas.
- ◆ El centroide, z_i es el representante de la clase



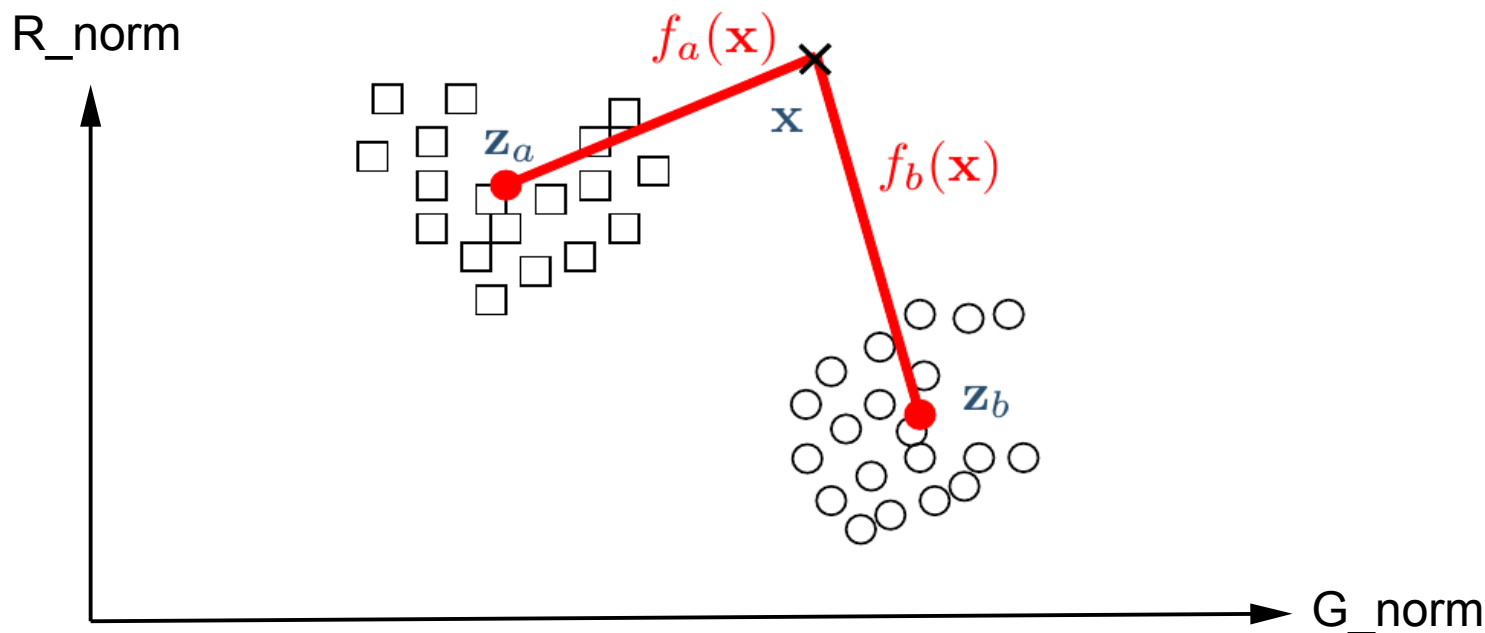
Clasificador de la distancia euclídea

■ Hipótesis.

- ◆ La dispersión de las clases es pequeña en relación a la distancia entre ellas.
- ◆ El centroide, z_i es el representante de la clase

■ Fundamentos.

- ◆ Asocio a cada clase una función de pertenencia
 $f_i(\mathbf{x}) \equiv d_E(\mathbf{x}, \mathbf{z}_i)$.



Clasificador de la distancia euclídea

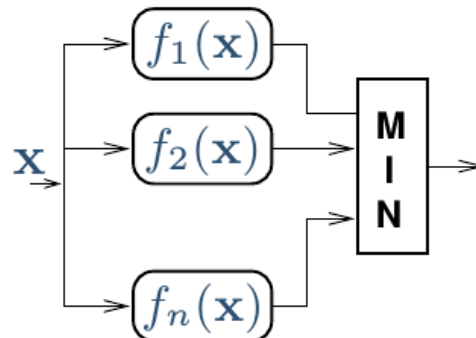
■ Hipótesis.

- ◆ La dispersión de las clases es pequeña en relación a la distancia entre ellas.
- ◆ El centroide, $\mathbf{z}_i$ es el representante de la clase

■ Fundamentos.

- ◆ Asocio a cada clase una función de pertenencia
 $f_i(\mathbf{x}) \equiv d_E(\mathbf{x}, \mathbf{z}_i)$.
- ◆ Clasifico un objeto $\mathbf{x}$ en la clase cuyo representante sea el más cercano $\mathbf{x} \in \alpha_i$ tal que $i = \arg \min_j \{f_j(\mathbf{x})\}$

■ Estructura.



Clasificación supervisada

- **Implementación sklearn (sugerida)**

Conjuntos de datos para entrenamiento supervisado:

X						y
	X[:, 0]	X[:, 1]	X[:, 2]	...	X[:, -1]	y
X[0]	1.23	4.75	0.54	...	1.45	0
X[1]	1.73	3.45	0.12	...	6.32	2
X[2]	0.32	8.21	1.11	...	9.99	2
...	...	...	...	...	...	1
...	...	...	...	...	...	0
X[-1]	4.56	3.24	6.83	...	6.43	1

Clasificación supervisada

● Implementación sklearn (sugerida)

Implementación de la interfaz de un clasificador:

```
class Classifier:
    @abstractmethod
    def fit(self, X, y):
        pass

    @abstractmethod
    def predict(self, X):
        pass

class ClassifEuclid(Classifier):
    def fit(self, X, y):
        """Entrena el clasificador
        X: matriz numpy, cada fila es un dato, cada columna una medida del vector de características.
        y: vector de clases, tantos elementos como filas en X.
        Retorna objeto clasificador"""
        """..."""
        return self

    def decision_function(self, X):
        """Estima el grado de pertenencia de todos los datos a todas las clase
        X: matriz numpy, cada fila es un dato, cada columna una medida del vector de características.
        Retorna una matriz, con tantas filas como datos y tantas columnas como clases tenga
        el problema, cada fila almacena los valores pertenencia de un dato a cada clase"""
        return """..."""

    def predict(self, X):
        """Predice una clase para cada dato. La clase retornada debe ser un entero.
        X: matriz numpy donde cada fila es un dato, cada columna una medida.
        Retorna un vector con la clase predicha para cada dato"""
        return """..."""
```

Clasificación supervisada

● Implementación

Implementación de la interfaz de un clasificador:

```
class Classifier:
    @abstractmethod
    def fit(self, X, y):
        pass

    @abstractmethod
    def predict(self, X):
        pass

class ClassifEuclid(Classifier):
    def fit(self, X, y):
        """Entrena el clasificador
        X: matriz numpy, cada fila es un dato, cada columna una medida del vector de características.
        y: vector de clases, tantos elementos como filas en X.
        Retorna objeto clasificador"""
        """..."""
        return self

    def decision_function(self, X):
        """Estima el grado de pertenencia de todos los datos a todas las clases
        X: matriz numpy, cada fila es un dato, cada columna una medida del vector de características.
        Retorna una matriz, con tantas filas como datos y tantas columnas como clases en el problema, cada fila almacena los valores de pertenencia de cada dato a cada clase"""
        return """..."""

    def predict(self, X):
        """Predice una clase para cada dato. La clase retornada debe ser un entero.
        X: matriz numpy donde cada fila es un dato, cada columna una medida.
        Retorna un vector con la clase predicha para cada dato"""
        return """..."""
```

Entradas:

X: (n_ejemplos, n_características)
y: (n_ejemplos,)

Entradas:

X: (n_ejemplos, n_características)
Salida:
(n_ejemplos, n_clases)

Entradas:

X: (n_ejemplos, n_características)
Salida:
(n_ejemplos,)

Clasificador de la distancia euclídea

● Implementación

`sklearn.neighbors.NearestCentroid`

```
class sklearn.neighbors.NearestCentroid(metric='euclidean', *, shrink_threshold=None)
```

[\[source\]](#)

Nearest centroid classifier.

Example:

```
>>> from sklearn.neighbors import NearestCentroid
>>> import numpy as np
>>> X = np.array([[-1, -1], [-2, -1], [-3, -2], [1, 1], [2, 1], [3, 2]])
>>> y = np.array([1, 1, 1, 2, 2, 2])
>>> clf = NearestCentroid()
>>> clf.fit(X, y)
NearestCentroid()
>>> print(clf.predict([[-0.8, -1]]))
[1]
```

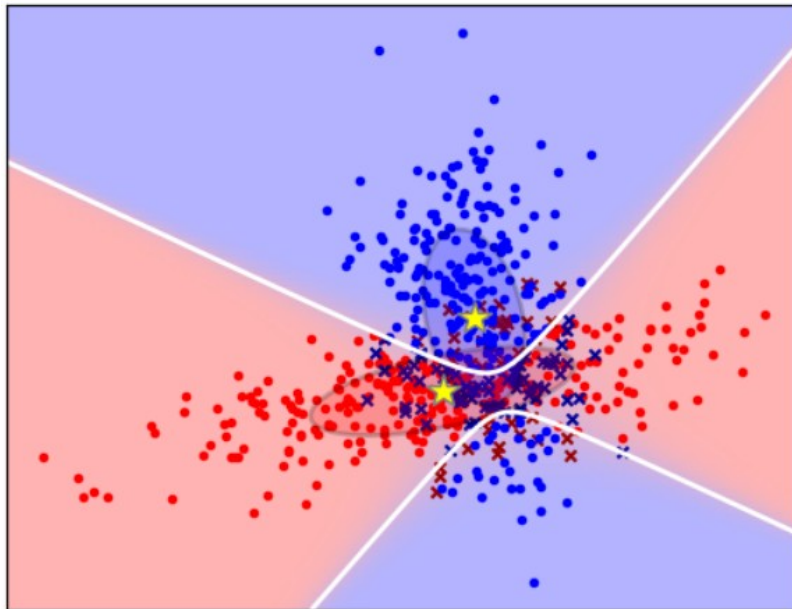
Clasificación supervisada

Sean $\mathcal{D} = \{(\mathbf{x}_1, c_1), \dots, (\mathbf{x}_n, c_k)\}$ los datos de entrenamiento con etiquetas $C = \{c_1, c_2, \dots\}$. Nuestro objetivo es aprender un modelo g tal que

$$c = g(\mathbf{x}|\mathcal{D})$$

prediga la etiqueta de cualquier otro objeto desconocido $\mathbf{x}$.

- **Clasificador Gaussiano**



Clasificador gaussiano

● Implementación

`sklearn.discriminant_analysis.QuadraticDiscriminantAnalysis`

```
class sklearn.discriminant_analysis.QuadraticDiscriminantAnalysis(*, priors=None, reg_param=0.0, store_covariance=False, tol=0.0001)
```

[\[source\]](#)

Quadratic Discriminant Analysis

A classifier with a quadratic decision boundary, generated by fitting class conditional densities to the data and using Bayes' rule.

The model fits a Gaussian density to each class.

Examples

```
>>> from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
>>> import numpy as np
>>> X = np.array([[ -1, -1], [ -2, -1], [ -3, -2], [ 1, 1], [ 2, 1], [ 3, 2]])
>>> y = np.array([1, 1, 1, 2, 2, 2])
>>> clf = QuadraticDiscriminantAnalysis()
>>> clf.fit(X, y)
QuadraticDiscriminantAnalysis()
>>> print(clf.predict([[ -0.8, -1]]))
[1]
```

Clasificación supervisada

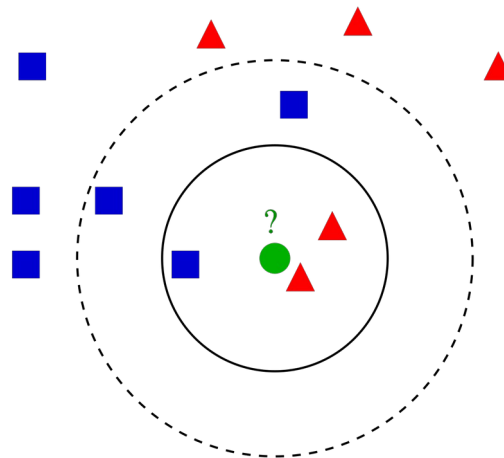
Sean $\mathcal{D} = \{(\mathbf{x}_1, c_1), \dots, (\mathbf{x}_n, c_k)\}$ los datos de entrenamiento con etiquetas $C = \{c_{hombre}, \dots, \}$. Nuestro objetivo es aprender un modelo g tal que

$$c = g(\mathbf{x}|\mathcal{D})$$

prediga la etiqueta de cualquier otro objeto desconocido $\mathbf{x}$.

En nuestro caso, los objetos a clasificar son los píxeles y/o marcas.

- **Clasificador del vecino más próximo**



Clasificación supervisada

Sean $\mathcal{D} = \{(\mathbf{x}_1, c_1), \dots, (\mathbf{x}_n, c_k)\}$ los datos de entrenamiento con etiquetas $C = \{c_{hombre}, \dots, \}$. Nuestro objetivo es aprender un modelo g tal que

$$c = g(\mathbf{x}|\mathcal{D})$$

prediga la etiqueta de cualquier otro objeto desconocido $\mathbf{x}$.

En nuestro caso, los objetos a clasificar son las marcas.

● Clasificador del vecino más próximo

Construimos $g(\mathbf{x}|\mathcal{D})$ directamente a partir de las muestras $\mathcal{D}$:

1. Calculo la distancia de $\mathbf{x}$ a todas las muestras de $\mathcal{D}$.
2. Sea $\mathbf{x}_j$ la muestra de $\mathcal{D}$ más cercana a $\mathbf{x}$.
3. Entonces asigno a $\mathbf{x}$ la etiqueta c_j de $\mathbf{x}_j$.

$$c_j = g(\mathbf{x}|\mathcal{D}) \equiv \{c_j \text{ tal que } j = \arg \min_k (\text{dist}(\mathbf{x}, \mathbf{x}_k))\}$$

Clasificación vecino más próximo

- Implementación

`sklearn.neighbors.KNeighborsClassifier`

```
class sklearn.neighbors.KNeighborsClassifier(n_neighbors=5, weights='uniform', algorithm='auto',  
leaf_size=30, p=2, metric='minkowski', metric_params=None, n_jobs=None, **kwargs) ¶ \[source\]
```

Classifier implementing the k-nearest neighbors vote.

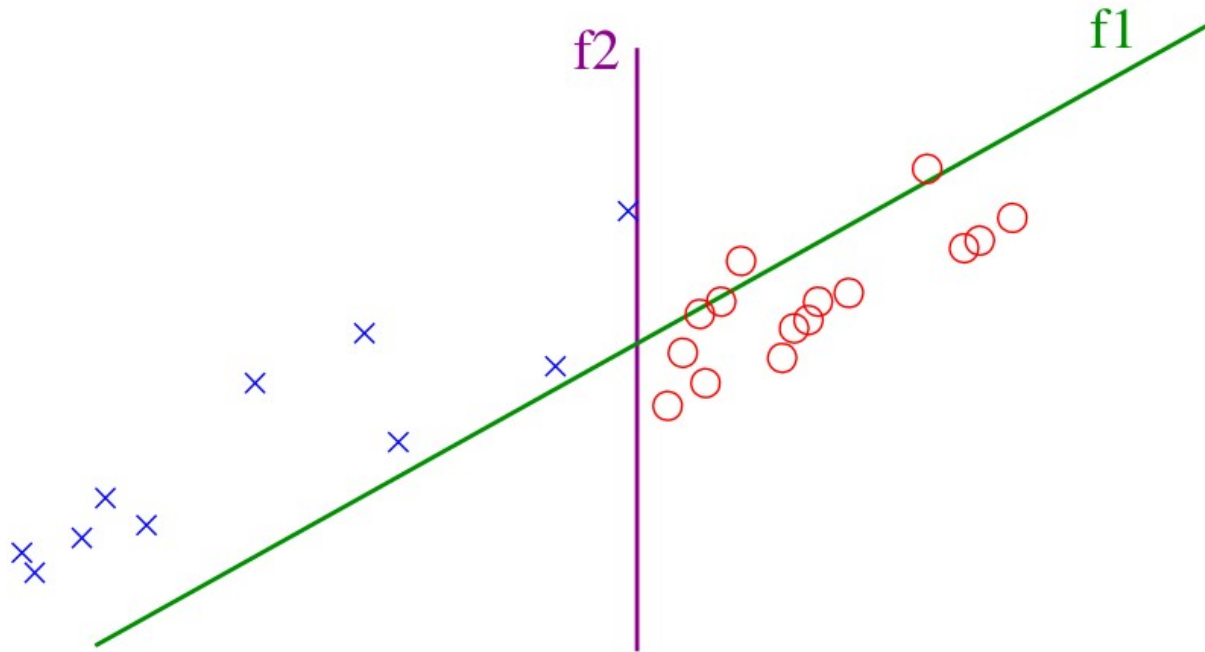
Examples

```
>>> X = [[0], [1], [2], [3]]  
>>> y = [0, 0, 1, 1]  
>>> from sklearn.neighbors import KNeighborsClassifier  
>>> neigh = KNeighborsClassifier(n_neighbors=3)  
>>> neigh.fit(X, y)  
KNeighborsClassifier(...)  
>>> print(neigh.predict([[1.1]]))  
[0]  
>>> print(neigh.predict_proba([[0.9]]))  
[[0.66666667 0.33333333]]
```


Clasificación supervisada evaluación

- **Objetivo**

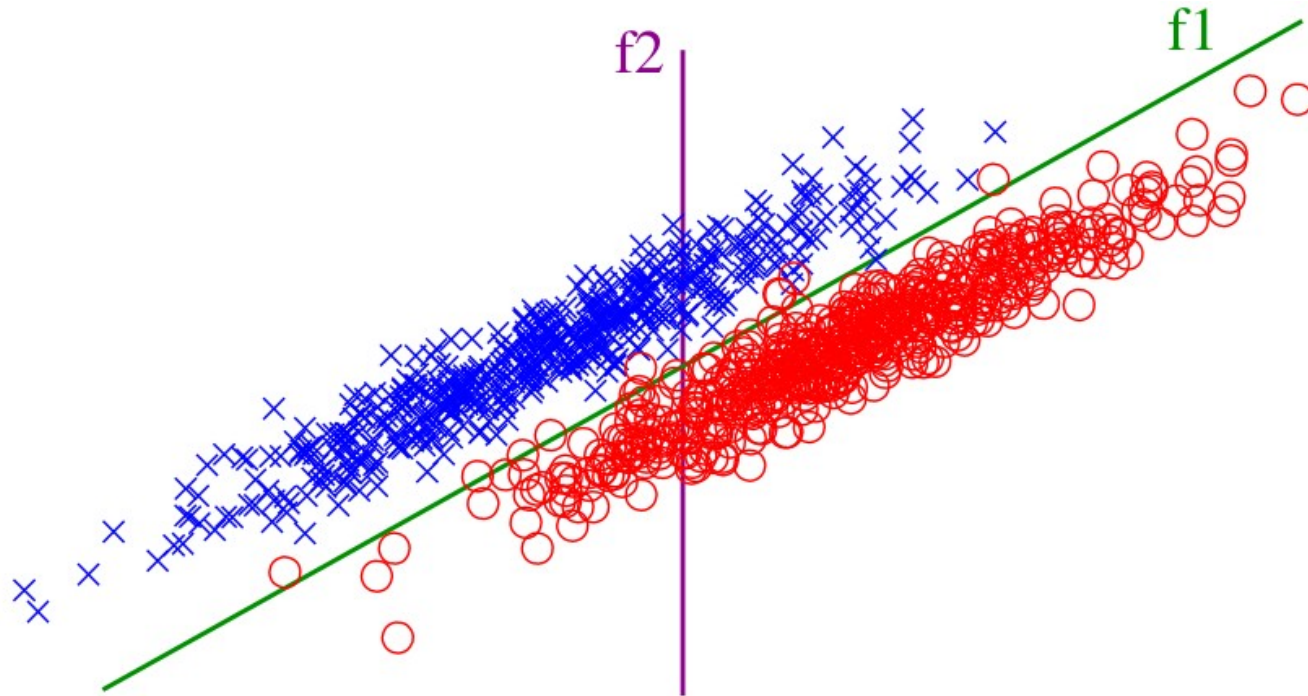
Estimar el error de generalización.



Clasificación supervisada evaluación

- **Objetivo**

Estimar el error de generalización.



El que obtendríamos al emplear el clasificador en muestras no utilizadas para entrenar.

Clasificación supervisada evaluación

- **Validación por exclusión**

Divide el conjunto de datos en 2 grupos

$$\mathcal{D} = \{\mathcal{E} \cup \mathcal{T}\}; \quad \mathcal{E} \cap \mathcal{T} = \emptyset$$

1. Entrena con los datos de entrenamiento $\mathcal{E}$
2. Evalúa con los datos de test $\mathcal{T}$

Típicamente

$$\text{card}(\mathcal{E}) = 0.8 \text{ card}(\mathcal{D})$$

Examples

```
>>> import numpy as np
>>> from sklearn.model_selection import train_test_split
>>> X, y = np.arange(10).reshape((5, 2)), range(5)
>>> X
array([[0, 1],
       [2, 3],
       [4, 5],
       [6, 7],
       [8, 9]])
>>> list(y)
[0, 1, 2, 3, 4]
```

```
>>> X_train, X_test, y_train, y_test = train_test_split(
...     X, y, test_size=0.33, random_state=42)
...
>>> X_train
array([[4, 5],
       [0, 1],
       [6, 7]])
>>> y_train
[2, 0, 3]
>>> X_test
array([[2, 3],
       [8, 9]])
>>> y_test
[1, 4]
```

Clasificación supervisada evaluación

- **Validación cruzada K-Fold**

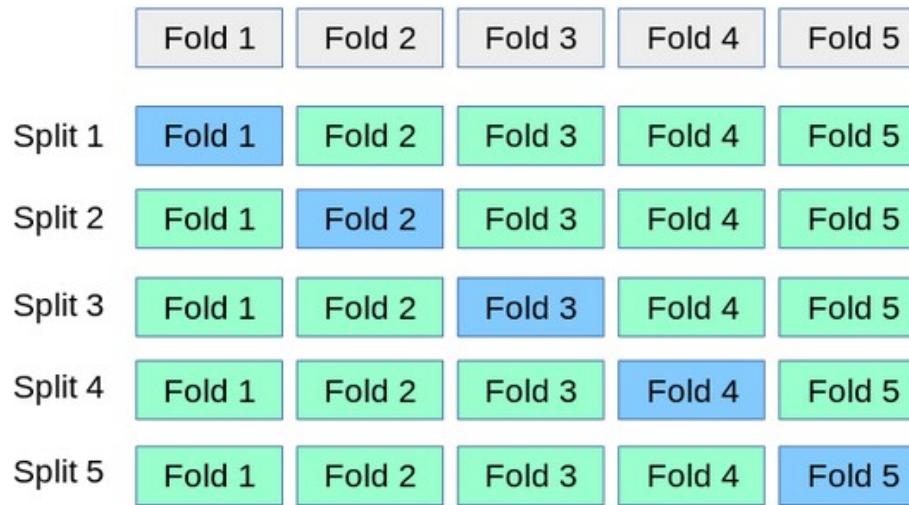
Divide el conjunto de datos en K grupos

1. Entrena con los datos de $K-1$ grupos

2. Evalúa con los datos del grupo excluido

Repite el proceso K veces

El resultado es el promedio de las K evaluaciones.



En validación cruzada Leave-One-Out (LOO)

$$K = \text{card}\{\mathcal{D}\}$$

Clasificación supervisada evaluación

- **Validación cruzada K-Fold**

Examples

```
>>> import numpy as np
>>> from sklearn.model_selection import KFold
>>> X = np.array([[1, 2], [3, 4], [1, 2], [3, 4]])
>>> y = np.array([1, 2, 3, 4])
>>> kf = KFold(n_splits=2)
>>> kf.get_n_splits(X)
2
>>> print(kf)
KFold(n_splits=2, random_state=None, shuffle=False)
>>> for train_index, test_index in kf.split(X):
...     print("TRAIN:", train_index, "TEST:", test_index)
...     X_train, X_test = X[train_index], X[test_index]
...     y_train, y_test = y[train_index], y[test_index]
TRAIN: [2 3] TEST: [0 1]
TRAIN: [0 1] TEST: [2 3]
```

Clasificación supervisada evaluación

- Validación cruzada Leave One Out

Examples

```
>>> import numpy as np
>>> from sklearn.model_selection import LeaveOneOut
>>> X = np.array([[1, 2], [3, 4]])
>>> y = np.array([1, 2])
>>> loo = LeaveOneOut()
>>> loo.get_n_splits(X)
2
>>> print(loo)
LeaveOneOut()
>>> for train_index, test_index in loo.split(X):
...     print("TRAIN:", train_index, "TEST:", test_index)
...     X_train, X_test = X[train_index], X[test_index]
...     y_train, y_test = y[train_index], y[test_index]
...     print(X_train, X_test, y_train, y_test)
TRAIN: [1] TEST: [0]
[[3 4]] [[1 2]] [2] [1]
TRAIN: [0] TEST: [1]
[[1 2]] [[3 4]] [1] [2]
```

Clasificación supervisada evaluación

- **Validación cruzada K-Fold**

Si hay problemas con la librería, la implementación del procedimiento LOO es trivial:

```
MiLOO = lambda max: iter([(np.array(range(0,i) + range(i+1,max)), np.array([i])) for i in range(max)])
```

```
for train, test in MiLOO(4):  
    print 'train: ', train  
    print 'test : ', test
```

```
train:  [1 2 3]  
test :  [0]  
train:  [0 2 3]  
test :  [1]  
train:  [0 1 3]  
test :  [2]  
train:  [0 1 2]  
test :  [3]
```


Clasificación supervisada evaluación

- **Validación cruzada K-Fold**

Si hay problemas con la librería, la implementación del procedimiento LOO es trivial:

```
MiLOO = lambda max: iter([(np.array(range(0,i) + range(i+1,max)), np.array([i])) for i in range(max)])
```

```
for train, test in MiLOO(4):  
    print 'train: ', train  
    print 'test : ', test
```

```
train:  [1 2 3]  
test :  [0]  
train:  [0 2 3]  
test :  [1]  
train:  [0 1 3]  
test :  [2]  
train:  [0 1 2]  
test :  [3]
```

```
X = np.array(['D1', 'D2', 'D3', 'D4'])  
y = np.array(['e1', 'e2', 'e3', 'e4'])  
for train, test in MiLOO(len(X)):  
    print 'train Datos: ', X[train]  
    print 'train Etiqs: ', y[train]  
    print 'test  Datos: ', X[test]  
    print 'test  Etiqs: ', y[test]
```

```
train Datos:  ['D2' 'D3' 'D4']  
train Etiqs:  ['e2' 'e3' 'e4']  
test  Datos:  ['D1']  
test  Etiqs:  ['e1']  
train Datos:  ['D1' 'D3' 'D4']  
train Etiqs:  ['e1' 'e3' 'e4']  
test  Datos:  ['D2']  
test  Etiqs:  ['e2']  
train Datos:  ['D1' 'D2' 'D4']  
train Etiqs:  ['e1' 'e2' 'e4']  
test  Datos:  ['D3']  
test  Etiqs:  ['e3']  
train Datos:  ['D1' 'D2' 'D3']  
train Etiqs:  ['e1' 'e2' 'e3']  
test  Datos:  ['D4']  
test  Etiqs:  ['e4']
```